Problem Set 9 (Optional)

Graded

20 Hours, 46 Minutes Late

Student

Elijah Martin McCauley

Total Points

97 / 100 pts

Question 1

Problem 1 (Dynamic Programming)

12 / 15 pts

– **0 pts** Correct

Table Definition (5 pts)

– **0 pts** Dimensions missing (no penalty)

– **1 pt** Minor mistake

– **3 pts** Mostly incorrect table definition.

– **5 pts** Table Definition Missing/Completely incorrect

Base Cases and Recurrence (10 pts)

– **1 pt** Minor mistake in base case/ Missing one base case

– **2 pts** Base Case Missing/Completely Incorrect

– **1 pt** Minor mistake in recurrence

– **3 pts** Major mistake in recurrence relation

– **5 pts** Recurrence Missing/Completely incorrect

✔ – **3 pts** Explanation of correctness Missing/incorrect

– **10 pts** Non-DP Approach

– **5 pts** Inefficient solution

– **15 pts** Missing Answer

Question 2

Problem 2 (Dynamic Programming)

5 / 5 pts

+ **0 pts** Incorrect

✔ + **5 pts** Correct (option c)

Question 3

Problem 3 (Dynamic Programming)

5 / 5 pts

+ **0 pts** Incorrect

✔ **+ 5 pts** Correct (option a)

Question 4

Problem 4 (Graphs)

20 / 20 pts

✔ **– 0 pts** Correct

– **4 pts** No graph definition provided/ Minor Error in Algorithm

– **10 pts** Major Error/ Mostly Incorrect/ Suboptimal Algorithm

– **5 pts** Pseudo-code provided, but algorithm is missing.

– **1 pt** Incorrect/ Missing Runtime Analysis

– **1 pt** Incorrect/ Missing Proof of Correctness

– **20 pts** Incorrect/ Missing answer

Question 5

Problem 5 (Graphs)

5 / 5 pts

+ **0 pts** Incorrect

✔ **+ 5 pts** Correct (option c)

Question 6

Problem 6 (Graphs)

5 / 5 pts

+ **0 pts** Incorrect

✔ **+ 5 pts** Correct (False)

Question 7

Problem 7 (Monte Carlo Methods)

10 / 10 pts

+ **0 pts** Incorrect

✔ **+ 10 pts** Correct (30.23%)

Question 8

Problem 8 (Big-O)

5 / 5 pts

+ **0 pts** Incorrect

✔ **+ 5 pts** Correct (option b)

Question 9

Problem 9 (D&C)

5 / 5 pts

+ **0 pts** Incorrect

✔ **+ 5 pts** Correct (option b)

Question 10

Problem 10 (D&C)

10 / 10 pts

✔ **– 0 pts** Correct

– **10 pts** Missing or incorrect algorithm

– **7 pts** Algorithm is mostly incorrect/ not using D&C

– **5 pts** Major Error/ Algorithm is not efficient/ partially correct

– **3 pts** Minor mistake in algorithm

– **3 pts** No justification/Only psuedocode provided

Question 11

Problem 11 (Complexity Theory)

5 / 5 pts

+ **0 pts** Incorrect

✔ **+ 5 pts** Correct (option c)

Question 12

Problem 12 (Complexity Theory)

5 / 5 pts

+ **0 pts** Incorrect

✔ **+ 5 pts** Correct (option c)

Question 13

Problem 13 (Complexity Theory)

5 / 5 pts

+ **0 pts** Incorrect

✔ **+ 5 pts** Correct (False)

Q1 Problem 1 (Dynamic Programming)
15 Points

Given a string of S of length N, we want to partition it into the minimum number of substrings such that each partition is a palindrome. Eg- Given a string $S = 01132$, a palindromic partitioning could be $[0, 1, 1, 3, 2]$, however this is not minimal, we can instead do $[0, 11, 3, 2]$ which gives us the answer 4. For the purpose of your solution, assume it takes $O(1)$ time to determine if any substring of the string is palindromic using the function $isPal(i, j)$ which returns whether $S_{i,i+1...j}$ is palindromic. Give the base cases, recurrence relation, and justification for a 1D state T_i.

The base cases are T0 is 0, T1 will be a one because the first letter in the string is a palindrome with itself. The recurrence relation is Ti = min(T[j-1] + 1) for 0 < j <= i such that S[j:i] is palindromic based on the function isPal(i, j). So for each index, we loop over j <= i and check if j to i is a palindrome in O(1) time. We then find the minimum of T[j-1] when j to i is palindromic because that is the number of palindromes before the current one containing i; this can be done by keeping a variable for the minimum value which can only be updated if a palindrome is found and the new T[j-1] + 1 is less than the previous minimum value (initialized to infinity). Basically, loop over i, for each j in 1 -> I, we check if it is a palindrome, if it is, we check T[j-1] + 1 < minimum palindromes, if it is we set minimum palindromes to T[j-1] + 1, after j = i, we set Ti to minimum palindromes. Ti is the fewest number of palindromes which can be made from S_0 to S_i.

Q2 Problem 2 (Dynamic Programming)
5 Points

Given two strings S and S' of lengths N and M respectively, to find the number of distinct subsequences (based on index chosen) of S that are equal to S', which of the following recurrences would we use

- ○ $T_{i,j} = T_{i-1,j}$
- ○ $T_{i,j} = T_{i-1,j} + T_{i-1,j-1}$
- ◉ $T_{i,j} = T_{i-1,j}$ if $S_i \neq S'_j$
  $T_{i,j} = T_{i-1,j} + T_{i-1,j-1}$ otherwise
- ○ $T_{i,j} = T_{i-1,j}$ if $S_i = S'_j$
  $T_{i,j} = T_{i-1,j} + T_{i-1,j-1}$ otherwise

Q3 Problem 3 (Dynamic Programming)
5 Points

Given a set of N types of items, where for each type i, you have a tuple $(value_i, count_i)$ which means you have $count_i$ items of type i each having value = $value_i$. Given this information, which of the following recurrences can be used to calculate the number of ways to get exactly $target$ value. Note that different items of the same type are identical so the only way two selections are considered different is if they don't use the same number of items for each of the types.

- ◉ $T_{i,j} = \sum T_{i-1,j-k*value_i}$ where $k * value_i \leq j$ and $0 \leq k \leq count_i$
- ○ $T_{i,j} = T_{i-1,j}$ if j < $value_i$
  $T_{i,j} = T_{i-1,j} + T_{i-1,j-value_i}$ otherwise

Q4 Problem 4 (Graphs)
20 Points

We place N balloons on a two-dimensional grid at integer coordinates. Each coordinate point can have at most one balloon.

A balloon can be popped if it shares either the same row or the same column as another balloon that has not been popped.

Given an array of N balloons where balloons$[i]$ = $[x_i, y_i]$ represents the location of the i^{th} balloon. Write an algorithm to determine the maximum number of balloons that can be popped.

Input: balloons = $[[2,0],[0,2],[1,1],[0,0],[2,2]]$
Output: 3
Explanation: One way to make three moves is as follows:
1. Pop balloon $[2,2]$ because it shares the same row as $[2,0]$.
2. Pop balloon $[2,0]$ because it shares the same column as $[0,0]$.
3. Pop balloon $[0,2]$ because it shares the same row as $[0,0]$.

Balloons $[0,0]$ and $[1,1]$ cannot be popped anymore since they do not share a row/column with another balloon still on the 2D grid.

We first need to create a graph given the input array. This can be done by looping over the array and creating nodes which correspond to the ballon locations, and putting edges between nodes if they share either a row or column. We set a variable balloons_popped to be 0. Once our graph is set up, we can run DFS on a random node which returns the number of balloons in that component. We know that we cannot pop all of the balloons in the component because there will be one balloon left which will no longer share a row or column, so we can say that we can pop N - 1 balloons where N is the size of the component and add that number to balloons_popped. We then go to the next node which has not been explored and do the same thing. After we have explored every node, return balloons_popped. This will run in O(|E|+|V|) because it is simply DFS.

Q5 Problem 5 (Graphs)
5 Points

Among the following, which one contributes to a valid tree?

- ○ A binary tree with a height of 6 and 65 leaves.
- ○ A tree with 10 vertices and 14 edges.
- ○ A binary tree with a height of 5 and 33 leaves.
- ◉ A rooted tree of height 3 with 40 total vertices, where every vertex has at most 3 children.
- ○ A full binary tree with 64 total vertices and a height of 5.

Q6 Problem 6 (Graphs)
5 Points

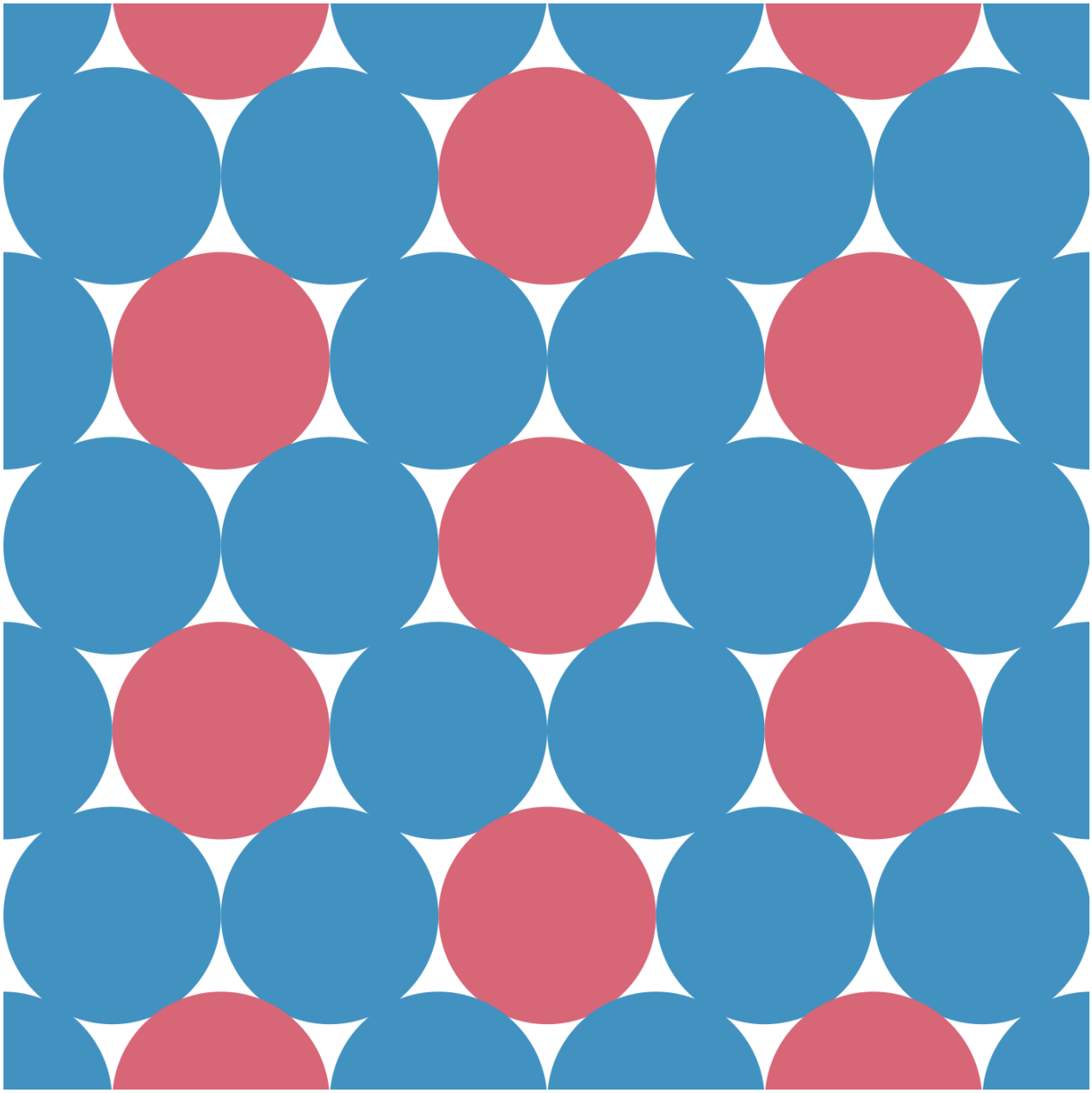
Dijkstra's algorithm can be used to find the longest path in a DAG by negating all the edge weights and running it from every source node.

- ○ True
- ◉ False

Q7 Problem 7 (Monte Carlo Methods)
10 Points

Your friends are all good at math, but you're not, so when you've all been presented with the following problem, you decide to write a simulation to find the solution.

On an infinite plane, there exist infinitely many circles. Each circle is of the same size and is packed densely such that each circle shares a tangent with six other circles. Consider the figure below. Find the percentage of area with respect to the size of the infinite plane which the circles take up.



Give the percentage of the area that is made up by red circles to two decimal places i.e., XX.XX%.

30.23%

Q8 Problem 8 (Big-O)
5 Points

Choose the correct relation for
$f(n) = n^{0.00001}, g(n) = (log(n))^3$

- $f(n) = O(g(n))$
- (selected) $g(n) = O(f(n))$
- $f(n) = O(g(n)) and g(n) = O(f(n))$

Q9 Problem 9 (D&C)
5 Points

Which of the following is the correct Big-O representation of the recurrence:
$T(n) = 64T(n/2) + 2^n$

- $O(logn)$
- (selected) $O(2^n)$
- $O(n^2)$
- $O(n)$

Q10 Problem 10 (D&C)
10 Points

You are given an Array $A[1....n]$ of sorted integers that has been circularly shifted k positions to the right. Your task is to find the largest element of A. Describe a $O(log(n))$ algorithm to accomplish this task. Your input is the array A only (i.e.: you don't know the value of k.) Describe your design in words and explain why it's correct.

Example: $A = [35, 42, 5, 15, 27, 29]$ has been circularly shifted $k = 2$ positions. Your algorithm should output 42.
Example: $A = [27, 29, 35, 42, 5, 15]$ has been circularly shifted $k = 4$ positions. Your algorithm should output 42.

You need to run a modified version of binary search where instead of comparing the midpoint of the current left and right pointers to a value you are looking for, you compare it to the value at the left pointer. If the value at the left pointer (which is initially at index 0) is greater than the midpoint, then the maximum value in the array is to the left, so we set the right pointer (which is initially set to index length - 1) to the current midpoint index. If the value at the left pointer is less than the value at the midpoint, we set the left pointer to the midpoint index. Once we have reached two elements remaining (so right - left = 1) we compare the value at the left index and the right index and return the larger of the two. This runs in O(log(n)) because each time we make a comparison, we halve the remaining elements we must look at so we make log(n) comparisons. Our algorithm works because if the midpoint is less than the left point, the greatest number has to be left of it, and if it is greater than the left point, we know the greatest number is either it or to the right of it.

Q11 Problem 11 (Complexity Theory)
5 Points

Which of the following statements about the complexity class P is false?

- ○ P is the set of decision problems that can be solved in polynomial time on a deterministic Turing machine.
- ○ P is a subset of the complexity class NP.
- ○ If a problem is in P, there exists a polynomial-time algorithm that solves it.
- ◉ P is the set of decision problems that can be solved in exponential time on a non-deterministic Turing machine.

Q12 Problem 12 (Complexity Theory)
5 Points

Which of the following statements about NP-complete problems is true?

- ○ NP-complete problems can be solved in polynomial time using a deterministic Turing machine.
- ○ If a problem is in NP-complete, it means that it is not in the complexity class NP.
- ◉ If an NP-complete problem can be solved in polynomial time, then all problems in NP can be solved in polynomial time.
- ○ NP-complete problems are a subset of the complexity class P.

Q13 Problem 13 (Complexity Theory)
5 Points

The worst-case running time of an algorithm is always equal to its average-case running time.

- ○ True
- ◉ False